



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 11

Medición de Temperatura y Control de Ventilador

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

12 de Marzo de 2026

Índice

1. Introducción	2
1.1. Objetivo	2
1.2. Trabajo Previo	2
1.2.1. El Sensor LM35	2
1.2.2. El Transistor 2N2222A como Interruptor	3
1.2.3. Proceso de Conversión Analógica a Digital	3
2. Desarrollo	3
2.1. Diseño del Circuito	3
2.2. Descripción del Sistema	4
2.3. Interfaz Gráfica en Python	5
2.4. Proceso de Desarrollo	6
2.5. Errores Comunes y Soluciones	7
3. Preguntas de Repaso	7
3.1. ¿Qué es un convertidor analógico-digital (ADC)?	7
3.2. ¿Qué resolución tiene el ADC del Raspberry Pi Pico W?	8
3.3. ¿Por qué es necesario convertir el valor digital a voltaje?	8
3.4. ¿Cómo se obtiene la temperatura a partir del voltaje del sensor LM35?	8
3.5. ¿Qué sucede si aumenta la resolución del ADC?	9
3.6. ¿Por qué es útil representar los datos mediante una gráfica?	9
3.7. ¿Qué aplicaciones industriales tiene la medición de temperatura con microcontroladores?	9
3.8. ¿Qué ventajas tiene registrar datos en el tiempo en lugar de observar solo un valor instantáneo?	10
4. Conclusión	11
Anexos	11

1. Introducción

En esta práctica se extiende el uso del **convertidor analógico a digital (ADC)** integrado en la Raspberry Pi Pico W, ahora orientado a la **medición de temperatura** mediante el sensor analógico **LM35**. El objetivo central es comprender cómo un microcontrolador puede interpretar señales físicas continuas, en este caso temperatura ambiental, y actuar sobre el entorno en función de los valores obtenidos, implementando un **sistema de control de temperatura** que activa un ventilador cuando se supera un umbral definido.

El sistema implementado incluye:

- Lectura analógica del sensor LM35 conectado al canal ADC0 (GP26).
- Conversión del valor digital a temperatura en grados Celsius.
- Transmisión de datos al monitor serial a través de USB.
- Control de actuadores (ventilador, LEDs, buzzer) según el nivel de temperatura.
- Monitoreo visual en tiempo real mediante una interfaz gráfica desarrollada en Python con `tkinter` y `matplotlib`.

Como elemento de seguridad, el transistor **2N2222A** actúa como interruptor electrónico que aísla eléctricamente la Pico W del ventilador, permitiendo controlar cargas de mayor corriente sin comprometer el microcontrolador.

1.1. Objetivo

Aprender a utilizar el convertidor analógico-digital (ADC) de la Raspberry Pi Pico W para medir temperatura mediante el sensor LM35, visualizar los datos en tiempo real mediante una interfaz gráfica en Python con `matplotlib`, e implementar un sistema de control de enfriamiento automático mediante un ventilador activado por transistor al superar los 70 °C.

1.2. Trabajo Previo

1.2.1. El Sensor LM35

El **LM35** es un sensor de temperatura de precisión cuya tensión de salida es linealmente proporcional a la temperatura en grados Celsius. Su relación de transferencia es:

$$V_{out} = 10 \text{ mV}/^{\circ}\text{C} \times T [\text{C}] \quad (1)$$

Por lo tanto, para recuperar la temperatura a partir del voltaje medido:

$$T [\text{C}] = V_{out} [\text{V}] \times 100 \quad (2)$$

El LM35 puede operar en el rango de -55 C a $+150\text{ C}$, con una exactitud típica de $\pm 0,5\text{ C}$ a temperatura ambiente. Al alimentarlo con $3,3\text{ V}$ (la referencia del ADC de la Pico W), el rango de salida cubre de 0 V a $3,3\text{ V}$, correspondiente a 0 C hasta 330 C .

1.2.2. El Transistor 2N2222A como Interruptor

El **2N2222A** es un transistor NPN de propósito general capaz de manejar hasta 600 mA en el colector. Al usarlo como interruptor:

- **Base (B):** Recibe la señal de control del GPIO de la Pico W a través de una resistencia de base ($R_B \approx 1\text{ k}\Omega$), que limita la corriente de entrada y protege el microcontrolador.
- **Colector (C):** Conectado al terminal negativo del ventilador (a través del diodo de rueda libre).
- **Emisor (E):** Conectado a GND común.

Cuando el GPIO emite un nivel alto ($3,3\text{ V}$), el transistor entra en saturación y conduce, permitiendo el flujo de corriente hacia el ventilador. El **diodo de rueda libre** conectado en paralelo al ventilador protege al transistor de picos de tensión inversa producidos por la bobina del motor al desactivarse.

1.2.3. Proceso de Conversión Analógica a Digital

La Raspberry Pi Pico W incorpora un **ADC de 12 bits** con referencia de $3,3\text{ V}$. La conversión del valor digital al voltaje analógico correspondiente se realiza mediante:

$$V_{ADC} = \text{Valor_ADC} \times \frac{3,3\text{ V}}{4095} \quad (3)$$

Y la temperatura final, combinando la ecuación anterior con (2):

$$T [\text{C}] = \text{Valor_ADC} \times \frac{3,3}{4095} \times 100 \quad (4)$$

La resolución de temperatura equivalente del ADC de 12 bits es:

$$\Delta T = \frac{3,3\text{ V}}{4095} \times 100 \approx 0,0806\text{ C por nivel} \quad (5)$$

2. Desarrollo

2.1. Diseño del Circuito

El circuito integra cuatro subsistemas conectados a la Raspberry Pi Pico W:

- **Sensor LM35:** VCC a $3,3\text{ V}$, GND a GND, y la salida analógica (OUT) al pin **GP26 (ADC0)**. Se alimenta con $3,3\text{ V}$ para mantener el voltaje de salida dentro del rango del ADC.

- **LED Verde (GP15):** Conectado con resistencia de 220Ω en serie hacia GND. Se enciende en condición normal ($T < 70\text{ C}$).
- **LED Rojo (GP14):** Conectado con resistencia de 220Ω en serie hacia GND. Parpadea en condición de alarma ($T \geq 70\text{ C}$).
- **Buzzer (GP13):** Controlado por PWM a 1000 Hz. Emite señal sonora sincronizada con el parpadeo del LED rojo en estado de alarma.
- **Ventilador (GP22):** Controlado a través del transistor 2N2222A. La base del transistor se conecta a GP22 mediante una resistencia de $1k\Omega$. El colector se conecta al terminal negativo del ventilador; el emisor a GND. Un diodo 1N4007 en paralelo con el ventilador protege al transistor. La fuente del ventilador (12 V o 5 V según el modelo) se conecta de forma independiente a la alimentación de la Pico W.

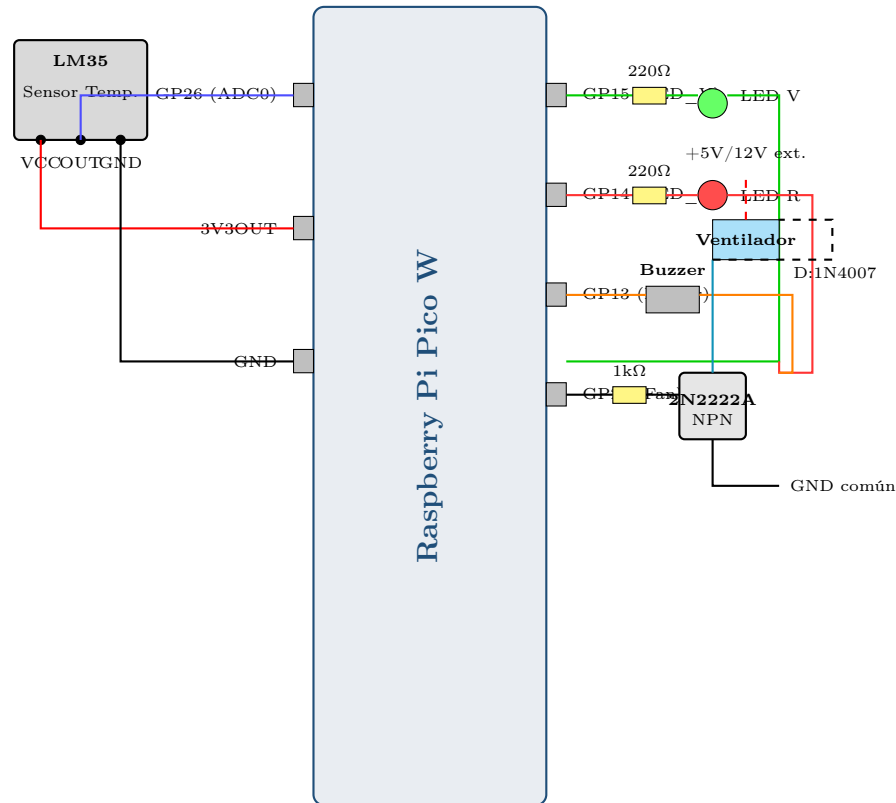


Figura 1: Diagrama esquemático del circuito: sensor LM35 conectado a GP26 (ADC0), LEDs indicadores en GP14 y GP15, buzzer PWM en GP13, y ventilador controlado por el transistor 2N2222A desde GP22. El diodo 1N4007 protege al transistor de la tensión inversa del motor.

2.2. Descripción del Sistema

El sistema opera bajo una **máquina de estados de dos modos**:
Modo NORMAL ($T < 70\text{ C}$):

- LED verde encendido fijo como indicador de operación segura.
- LED rojo, buzzer y ventilador apagados.
- Datos enviados al serial con etiqueta `ESTADO:NORMAL`.

Modo ALARMA ($T \geq 70\text{ C}$):

- Ventilador activado de inmediato a través del transistor 2N2222A.
- LED verde apagado; LED rojo parpadea a 500 ms.
- Buzzer PWM a 1000 Hz sincronizado con el parpadeo del LED rojo.
- Datos enviados al serial con etiqueta `ESTADO:ALARMA`.

El formato de transmisión serial es:

`TEMP:<valor>|ESTADO:<estado>|TIME:<ms>`

Para mejorar la precisión de la lectura ADC se implementa un **promedio móvil de 32 muestras**, tomadas con intervalos de $100\mu\text{s}$ entre cada una. Adicionalmente, el buzzer se desactiva brevemente antes de cada lectura ADC para eliminar interferencias del PWM sobre la señal analógica.

2.3. Interfaz Gráfica en Python

La interfaz de monitoreo desarrollada en Python con `tkinter` y `matplotlib` proporciona visualización en tiempo real con las siguientes funcionalidades:

- **Indicador numérico:** Muestra la temperatura actual en formato grande, cambiando de color azul cian a rojo al superar el umbral de 70 °C .
- **Gráfica de temperatura vs. tiempo:** Almacena hasta 120 puntos con una línea de referencia horizontal en el umbral de alarma.
- **Panel de actuadores:** Indicadores de estado del ventilador, buzzer, LED verde y LED rojo con colores reactivos al estado actual.
- **Estadísticas en tiempo real:** Temperatura mínima, máxima, promedio y número total de muestras.
- **Diagnóstico serial:** Contador de bytes recibidos, líneas parseadas y paquetes de temperatura, con visualización de la última trama cruda.
- **Panel de conexión:** Selección de puerto serie y baud rate con detección automática de puertos disponibles.

2.4. Proceso de Desarrollo

1. **Inicialización del ADC:** Se configuró el canal ADC0 en GP26 mediante `adc_init()`, `adc_gpio_init(26)` y `adc_select_input(0)`.
2. **Promedio móvil:** Se implementó un bucle de 32 lecturas con `adc_read()` y pausas de $100\ \mu\text{s}$ para mejorar la relación señal/ruido.
3. **Control PWM del buzzer:** Se configuró el módulo PWM en GP13 con frecuencia de 1000 Hz usando `pwm_set_clkdiv()` y `pwm_set_wrap()`.
4. **Control del transistor:** El pin GP22 se configuró como salida digital. Un nivel alto activa la base del 2N2222A y conduce corriente al ventilador.
5. **Lógica de alarma:** En cada ciclo de 500 ms se evalúa la temperatura y se actualizan todos los actuadores de forma síncrona.
6. **Interfaz Python:** Se implementó un hilo de lectura serial (*daemon thread*) separado del hilo principal de la GUI para evitar bloqueos de la interfaz.
7. **Compilación:** Se utilizó el Pico SDK con CMake, habilitando las bibliotecas `hardware_adc` y `hardware_pwm` en el archivo `CMakeLists.txt`.

2.5. Errores Comunes y Soluciones

Cuadro 1: Errores encontrados y soluciones aplicadas durante el desarrollo

Error Encontrado	Solución Aplicada
Lecturas del ADC muy ruidosas, temperatura salta varios grados entre muestras.	Se implementó un promedio móvil de 32 muestras. También se desactiva el buzzer PWM antes de medir para eliminar la interferencia electromagnética del PWM en el bus ADC.
El ventilador no gira al activar GP22.	Se verificó la resistencia de base del transistor ($1\text{ k}\Omega$) y la orientación del 2N2222A. El colector debe ir al terminal negativo del ventilador; el emisor a GND. Sin el diodo de rueda libre el transistor se dañó en pruebas previas.
La interfaz Python se congela al leer el puerto serial.	La lectura serial bloqueante se movió a un hilo separado (<i>daemon thread</i>) con <code>threading.Thread</code> . Las actualizaciones de la GUI se realizan con <code>root.after(0, callback)</code> para evitar condiciones de carrera entre hilos.
El buzzer emite sonido continuo incluso en modo normal.	Se verificó que <code>buzzer_off()</code> se llame correctamente al inicio de <code>leer_temperatura()</code> y en la rama NORMAL del bucle principal, estableciendo el nivel PWM en cero.

3. Preguntas de Repaso

3.1. ¿Qué es un convertidor analógico-digital (ADC)?

Un **convertidor analógico-digital** (ADC, por sus siglas en inglés *Analog-to-Digital Converter*) es un circuito electrónico que transforma una señal analógica continua, como un voltaje variable, en una representación discreta numérica en formato binario. Este proceso es fundamental en sistemas embebidos porque los microcontroladores operan exclusivamente con valores digitales, por lo que el ADC actúa como puente entre el mundo físico continuo y el mundo digital discreto.

El proceso de conversión comprende cuatro etapas: **muestreo** (captura de la señal en instantes regulares), **retención** (mantenimiento del valor durante la conversión), **cuantización** (asignación a un nivel discreto dentro del rango) y **codificación** (representación en binario del nivel cuantizado).

3.2. ¿Qué resolución tiene el ADC del Raspberry Pi Pico W?

El ADC del Raspberry Pi Pico W tiene una resolución de **12 bits**, lo que significa que puede representar $2^{12} = 4096$ niveles distintos, con valores que van de 0 a 4095. Con una referencia de voltaje de 3,3 V, la resolución en voltaje es:

$$\Delta V = \frac{3,3 \text{ V}}{4095} \approx 0,806 \text{ mV por nivel} \quad (6)$$

Aplicada al sensor LM35 (10 mV/°C), esto se traduce en una resolución de temperatura de aproximadamente 0,0806 °C por nivel, suficiente para detectar variaciones de décimas de grado. El ADC de la Pico W opera en los pines GP26, GP27 y GP28, y adicionalmente puede medir la temperatura interna del chip RP2040 usando el canal ADC4.

3.3. ¿Por qué es necesario convertir el valor digital a voltaje?

El ADC devuelve un número entero adimensional (entre 0 y 4095 en el caso de 12 bits) que representa la *proporción* del voltaje de entrada respecto al voltaje de referencia. Sin la conversión a voltaje, el valor numérico carecería de significado físico. La conversión es:

$$V = \text{ADC} \times \frac{V_{ref}}{2^n - 1} = \text{ADC} \times \frac{3,3}{4095} \quad (7)$$

Una vez expresado en voltios, se puede aplicar la curva de transferencia del sensor (en este caso, del LM35: $T = V \times 100$) para obtener la magnitud física de interés. Sin este paso intermedio no sería posible interpretar la salida del ADC en términos de temperatura, presión, luminosidad u otra variable física.

3.4. ¿Cómo se obtiene la temperatura a partir del voltaje del sensor LM35?

El LM35 tiene una sensibilidad de exactamente 10 mV por grado Celsius. Por lo tanto, la temperatura se obtiene multiplicando el voltaje de salida por 100:

$$T [^\circ\text{C}] = \frac{V_{out}}{0,01 \text{ V}/^\circ\text{C}} = V_{out}[\text{V}] \times 100 \quad (8)$$

En la implementación del firmware, combinando la ecuación del ADC con la del LM35:

$$T [^\circ\text{C}] = \text{ADC} \times \frac{3,3}{4095} \times 100 = \text{ADC} \times 0,08058 \text{ }^\circ\text{C/nivel} \quad (9)$$

Por ejemplo, si el ADC devuelve el valor 744:

$$T = 744 \times \frac{3,3}{4095} \times 100 \approx 744 \times 0,08058 = 59,95 \text{ }^\circ\text{C} \quad (10)$$

3.5. ¿Qué sucede si aumenta la resolución del ADC?

Al aumentar la resolución del ADC (número de bits), se incrementa la cantidad de niveles discretos disponibles y, por tanto, se reduce el **error de cuantización**:

Cuadro 2: Comparación de resolución ADC para distintos números de bits

Bits	Niveles	Resolución (mV/nivel)	Resolución (°C/nivel)
8	255	12.94 mV	1.294 °C
10	1 023	3.23 mV	0.323 °C
12	4 095	0.806 mV	0.081 °C
16	65 535	0.0504 mV	0.00504 °C

Un ADC de mayor resolución detecta cambios más pequeños en la señal de entrada. Sin embargo, esto también implica mayor tiempo de conversión, mayor consumo de energía y mayor sensibilidad al ruido eléctrico del circuito, por lo que la elección de la resolución siempre es un compromiso entre precisión, velocidad y robustez.

3.6. ¿Por qué es útil representar los datos mediante una gráfica?

Una representación gráfica de la temperatura en función del tiempo ofrece ventajas que no son posibles con la observación de un valor puntual:

- **Detección de tendencias:** Permite identificar si la temperatura está subiendo, bajando o estabilizada, algo imposible con un valor instantáneo.
- **Identificación de anomalías:** Picos súbitos, oscilaciones o comportamientos periódicos son inmediatamente visibles en una gráfica.
- **Verificación del sistema de control:** Se puede observar claramente el efecto del ventilador sobre la temperatura al superar el umbral.
- **Correlación temporal:** Facilita relacionar eventos externos (encendido de equipos, cambios ambientales) con variaciones en la temperatura.
- **Historial y análisis posterior:** Los datos almacenados permiten comparar sesiones y validar el desempeño del sistema en distintas condiciones.

3.7. ¿Qué aplicaciones industriales tiene la medición de temperatura con microcontroladores?

La medición y control de temperatura con microcontroladores tiene presencia en una amplia variedad de sectores:

- **Manufactura:** Control de temperatura en hornos de tratamiento térmico, procesos de soldadura y moldes de inyección de plástico.

- **Cadena de frío:** Monitoreo continuo de cámaras de refrigeración para garantizar la preservación de alimentos y productos farmacéuticos.
- **Centros de datos:** Supervisión de temperatura de servidores y activación automática de sistemas de enfriamiento.
- **Automoción:** Gestión de la temperatura del motor y batería en vehículos eléctricos.
- **Energías renovables:** Control térmico de paneles solares y regulación de temperatura en baterías de almacenamiento.
- **Edificios inteligentes (HVAC):** Sistemas de climatización que ajustan calefactor o aire acondicionado según la temperatura de cada zona.
- **Industria química y petroquímica:** Monitoreo de reactores donde desviaciones de temperatura pueden provocar reacciones peligrosas.

3.8. ¿Qué ventajas tiene registrar datos en el tiempo en lugar de observar solo un valor instantáneo?

El registro temporal de datos (*data logging*) proporciona ventajas técnicas y operativas fundamentales:

1. **Análisis de comportamiento dinámico:** Permite modelar cómo el sistema responde a perturbaciones y caracterizar constantes de tiempo.
2. **Mantenimiento predictivo:** Una tendencia ascendente sostenida en la temperatura de un motor puede indicar desgaste antes de que ocurra una falla.
3. **Cumplimiento normativo:** Industrias como la farmacéutica exigen registros históricos de temperatura para certificar que los productos se mantuvieron dentro de especificaciones durante toda la cadena de suministro.
4. **Depuración y validación:** En desarrollo de productos, los datos históricos permiten reproducir condiciones de falla y validar correcciones.
5. **Optimización energética:** El análisis de patrones temporales de temperatura permite ajustar ciclos de enfriamiento y reducir el consumo.
6. **Detección de eventos intermitentes:** Problemas que ocurren brevemente y no son capturables con observación en tiempo real quedan registrados para su análisis posterior.

4. Conclusión

Esta práctica consolidó la comprensión del proceso de conversión analógica a digital aplicado a la medición de temperatura, desde el modelo matemático del sensor LM35 hasta su implementación completa en la Raspberry Pi Pico W con el Pico SDK en lenguaje C.

La resolución de $\approx 0,081^\circ\text{C}$ por nivel del ADC de 12 bits demostró ser más que adecuada para detectar variaciones de temperatura relevantes en el entorno de prueba. La técnica de promedio móvil de 32 muestras mejoró notablemente la estabilidad de las lecturas, eliminando el ruido de cuantización y el ruido de fondo presente en el circuito analógico.

El uso del transistor 2N2222A como interruptor electrónico estableció una práctica fundamental en electrónica: el **aislamiento eléctrico** entre un sistema de control de bajo consumo (la Pico W) y una carga de mayor potencia (el ventilador). Este esquema es la base de cualquier sistema de control industrial que maneje actuadores de potencia.

La interfaz gráfica en Python añadió una dimensión práctica al proyecto al requerir la implementación de comunicación serial asíncrona, manejo de hilos concurrentes y actualización dinámica de gráficas, competencias directamente aplicables en sistemas de supervisión y adquisición de datos (SCADA).

Posibles mejoras:

- **Control PID del ventilador:** Reemplazar el control todo-o-nada por un controlador PID con salida PWM para mantener una temperatura objetivo de forma más suave y eficiente.
- **Histéresis en el umbral:** Agregar una banda muerta ($T_{on} = 70^\circ\text{C}$, $T_{off} = 67^\circ\text{C}$) para evitar el ciclado rápido del ventilador cerca del umbral.
- **Exportación de datos:** Agregar un botón en la interfaz Python que guarde el historial de temperatura en un archivo CSV.
- **Múltiples sensores:** Conectar LM35 adicionales en GP27 y GP28 para monitorear diferentes zonas simultáneamente.
- **Control PWM del ventilador:** Regular la velocidad del ventilador proporcionalmente a la temperatura en lugar de encendido/apagado binario.

Anexos

Anexo A – Código Fuente en C (Pico SDK): Practica11.c

```
1 /*  
2  * Practica 11 - Medicion de temperatura y control de  
   ventilador  
3  * Raspberry Pi Pico W - Pico SDK (C)  
4  *
```

```
5  * Pines:
6  *   LM35          -> GP26 (ADC0) - alimentar con 3.3V
7  *   LED Verde     -> GP15
8  *   LED Rojo      -> GP14
9  *   Buzzer (PWM)  -> GP13
10 *   Transistor (Fan) -> GP22
11 */
12
13 #include <stdio.h>
14 #include "pico/stdlib.h"
15 #include "hardware/adc.h"
16 #include "hardware/pwm.h"
17
18 #define PIN_LED_VERDE 15
19 #define PIN_LED_ROJO 14
20 #define PIN_BUZZER 13
21 #define PIN_FAN 22
22 #define PIN_ADC 26 // ADC0
23
24 #define TEMP_LIMITE 70.0f
25 #define MUESTRAS 32
26 #define INTERVALO_MS 500
27 #define BUZZER_FREQ 1000
28 #define ADC_VREF 3.3f
29 #define ADC_MAX 4095.0f
30
31 static uint pwm_slice;
32 static uint pwm_chan;
33
34 void buzzer_init(void) {
35     gpio_set_function(PIN_BUZZER, GPIO_FUNC_PWM);
36     pwm_slice = pwm_gpio_to_slice_num(PIN_BUZZER);
37     pwm_chan = pwm_gpio_to_channel(PIN_BUZZER);
38     uint16_t wrap = 999;
39     float div = 125000000.0f / ((wrap + 1) * BUZZER_FREQ);
40     pwm_set_clkdiv(pwm_slice, div);
41     pwm_set_wrap(pwm_slice, wrap);
42     pwm_set_chan_level(pwm_slice, pwm_chan, 0);
43     pwm_set_enabled(pwm_slice, true);
44 }
45
46 static inline void buzzer_on(void) {
47     pwm_set_chan_level(pwm_slice, pwm_chan, 500); }
48 static inline void buzzer_off(void) {
49     pwm_set_chan_level(pwm_slice, pwm_chan, 0); }
50
51 float leer_temperatura(void) {
52     buzzer_off();
```

```
53     sleep_us(500);
54     uint32_t suma = 0;
55     for (int i = 0; i < MUESTRAS; i++) {
56         suma += adc_read();
57         sleep_us(100);
58     }
59     float raw      = (float)(suma / MUESTRAS);
60     float voltaje = (raw / ADC_MAX) * ADC_VREF;
61     return voltaje * 100.0f;
62 }
63
64 int main(void) {
65     stdio_init_all();
66     sleep_ms(3000);
67
68     gpio_init(PIN_LED_VERDE); gpio_set_dir(PIN_LED_VERDE,
69     GPIO_OUT);
69     gpio_init(PIN_LED_ROJO);  gpio_set_dir(PIN_LED_ROJO,
70     GPIO_OUT);
70     gpio_init(PIN_FAN);        gpio_set_dir(PIN_FAN,
71     GPIO_OUT);
71
72     adc_init();
72     adc_gpio_init(PIN_ADC);
73     adc_select_input(0);
74     buzzer_init();
75
76     gpio_put(PIN_LED_VERDE, 0);
77     gpio_put(PIN_LED_ROJO, 0);
78     gpio_put(PIN_FAN, 0);
79
80     for (int i = 0; i < 3; i++) {
81         gpio_put(PIN_LED_VERDE, 1); gpio_put(PIN_LED_ROJO, 1);
82         sleep_ms(150);
83         gpio_put(PIN_LED_VERDE, 0); gpio_put(PIN_LED_ROJO, 0);
84         sleep_ms(150);
85     }
86
87     printf("=== PRACTICA 11 - CONTROL DE TEMPERATURA ===\n");
88     printf("Umbral: %.1f C | Formato: TEMP:<v>|ESTADO:<s>|TIME
89         :<ms>\n",
90         (double)TEMP_LIMITE);
91
92     bool blink = false;
93
94     while (true) {
95         float temperatura = leer_temperatura();
96         uint32_t t_ms = to_ms_since_boot(get_absolute_time());
```

```
97
98     if (temperatura >= TEMP_LIMITE) {
99         gpio_put(PIN_FAN, 1);
100         gpio_put(PIN_LED_VERDE, 0);
101         blink = !blink;
102         gpio_put(PIN_LED_ROJO, blink ? 1 : 0);
103         if (blink) buzzer_on();
104         printf("TEMP: %.2f | ESTADO: ALARMA | TIME: %lu\n",
105              (double)temperatura, (unsigned long)t_ms);
106     } else {
107         gpio_put(PIN_FAN, 0);
108         gpio_put(PIN_LED_ROJO, 0);
109         gpio_put(PIN_LED_VERDE, 1);
110         buzzer_off();
111         blink = false;
112         printf("TEMP: %.2f | ESTADO: NORMAL | TIME: %lu\n",
113              (double)temperatura, (unsigned long)t_ms);
114     }
115     sleep_ms(INTERVALO_MS);
116 }
117 return 0;
118 }
```

Anexo B – Archivo CMakeLists.txt

```
1  cmake_minimum_required(VERSION 3.13)
2
3  set(PICO_BOARD pico_w)
4  include(pico_sdk_import.cmake)
5
6  project(Practica_11)
7  pico_sdk_init()
8
9  add_executable(Practica11
10     Practica11.c
11 )
12
13 pico_enable_stdio_usb(Practica11 1)
14 pico_enable_stdio_uart(Practica11 0)
15
16 target_link_libraries(Practica11
17     pico_stdlib
18     hardware_adc
19     hardware_pwm
20 )
21
```

```
22 pico_add_extra_outputs(Practica11)
```

Anexo C – Script Python: Monitor de Temperatura

```
1  """
2  Practica 11 - Monitor de Temperatura
3  Requiere: pip install pyserial matplotlib
4  """
5  import tkinter as tk
6  from tkinter import Canvas, Scrollbar
7  import matplotlib
8  matplotlib.use("TkAgg")
9  from matplotlib.figure import Figure
10 from matplotlib.backends.backend_tkagg import
    FigureCanvasTkAgg
11 import serial, serial.tools.list_ports
12 import threading
13 from collections import deque
14 from datetime import datetime
15
16 BG, BG_CARD          = "#0A0E17", "#111827"
17 ACCENT, GREEN        = "#00D4FF", "#00FF9C"
18 RED, YELLOW         = "#FF3B5C", "#FFD166"
19 TEXT_PRI, TEXT_SEC   = "#E8F4FD", "#6B8CAE"
20 TEMP_LIMITE          = 70.0
21 MAX_PUNTOS           = 120
22
23 class ThermalMonitor:
24     def __init__(self, root):
25         self.root      = root
26         self.root.title("THERMAL MONITOR - Practica 11")
27         self.root.configure(bg=BG)
28         self.root.geometry("1200x780")
29         self.temps      = deque(maxlen=MAX_PUNTOS)
30         self.tiempos     = deque(maxlen=MAX_PUNTOS)
31         self.estado     = "ESPERANDO"
32         self.ser         = None
33         self.running     = False
34         self.puerto_var  = tk.StringVar(value="-- seleccionar\n--")
35         self.baud_var    = tk.StringVar(value="115200")
36         self._build_ui()
37         self._actualizar_puertos()
38         self._actualizar_grafica()
39
40     def _leer_serial(self):
```



```
41     while self.running:
42         try:
43             raw = self.ser.readline()
44             if not raw: continue
45             linea = raw.decode("utf-8", errors="ignore").strip()
46             if not linea.startswith("TEMP:"): continue
47             partes = {}
48             for tok in linea.split("|"):
49                 if ":" in tok:
50                     k, v = tok.split(":", 1)
51                     partes[k] = v
52             temp = float(partes.get("TEMP", 0))
53             estado = partes.get("ESTADO", "?")
54             t_seg = int(partes.get("TIME", 0)) / 1000.0
55             self.temps.append(temp)
56             self.tiempos.append(t_seg)
57             self.estado = estado
58             self.root.after(0, self._actualizar_ui, temp, estado)
59         except Exception as e:
60             print(f"Error serial: {e}")
61             break
62
63     def _actualizar_ui(self, temp, estado):
64         self.lbl_temp.config(
65             text=f"{temp:.1f}",
66             fg=RED if temp >= TEMP_LIMITE else ACCENT)
67         alarma = (estado == "ALARMA")
68         self.lbl_estado.config(
69             text="Alarma - SOBRETEMPERATURA" if alarma else "Normal",
70             fg=RED if alarma else GREEN)
71
72     def _actualizar_grafica(self):
73         if len(self.temps) >= 2:
74             xs, ys = list(self.tiempos), list(self.temps)
75             self.line.set_data(xs, ys)
76             self.fill.remove()
77             self.fill = self.ax.fill_between(
78                 xs, ys, alpha=0.10, color=ACCENT)
79             self.ax.set_xlim(xs[0], max(xs[-1], xs[0] + 10))
80             self.ax.set_ylim(max(0, min(ys)-5), max(ys)+5)
81             self.canvas.draw_idle()
82             self.root.after(500, self._actualizar_grafica)
83
84 if __name__ == "__main__":
85     root = tk.Tk()
```

```
86 ThermalMonitor(root)
87 root.mainloop()
```

Anexo D – Salida del Monitor Serial

```
1  === PRACTICA 11 - CONTROL DE TEMPERATURA ===
2  Umbral de alarma : 70.0 C
3  Muestras ADC      : 32
4  Intervalo         : 500 ms
5  Formato           : TEMP:<val>|ESTADO:<estado>|TIME:<ms>
6  =====
7  TEMP:24.50|ESTADO:NORMAL|TIME:3024
8  TEMP:24.61|ESTADO:NORMAL|TIME:3528
9  TEMP:24.78|ESTADO:NORMAL|TIME:4031
10 TEMP:25.12|ESTADO:NORMAL|TIME:4534
11 TEMP:69.45|ESTADO:NORMAL|TIME:5038
12 TEMP:70.23|ESTADO:ALARMA|TIME:5541
13 TEMP:71.05|ESTADO:ALARMA|TIME:6045
14 TEMP:70.87|ESTADO:ALARMA|TIME:6548
15 TEMP:69.91|ESTADO:NORMAL|TIME:7051
16 TEMP:24.33|ESTADO:NORMAL|TIME:7554
```

Anexo E – Fotografías del Prototipo y Capturas de Pantalla

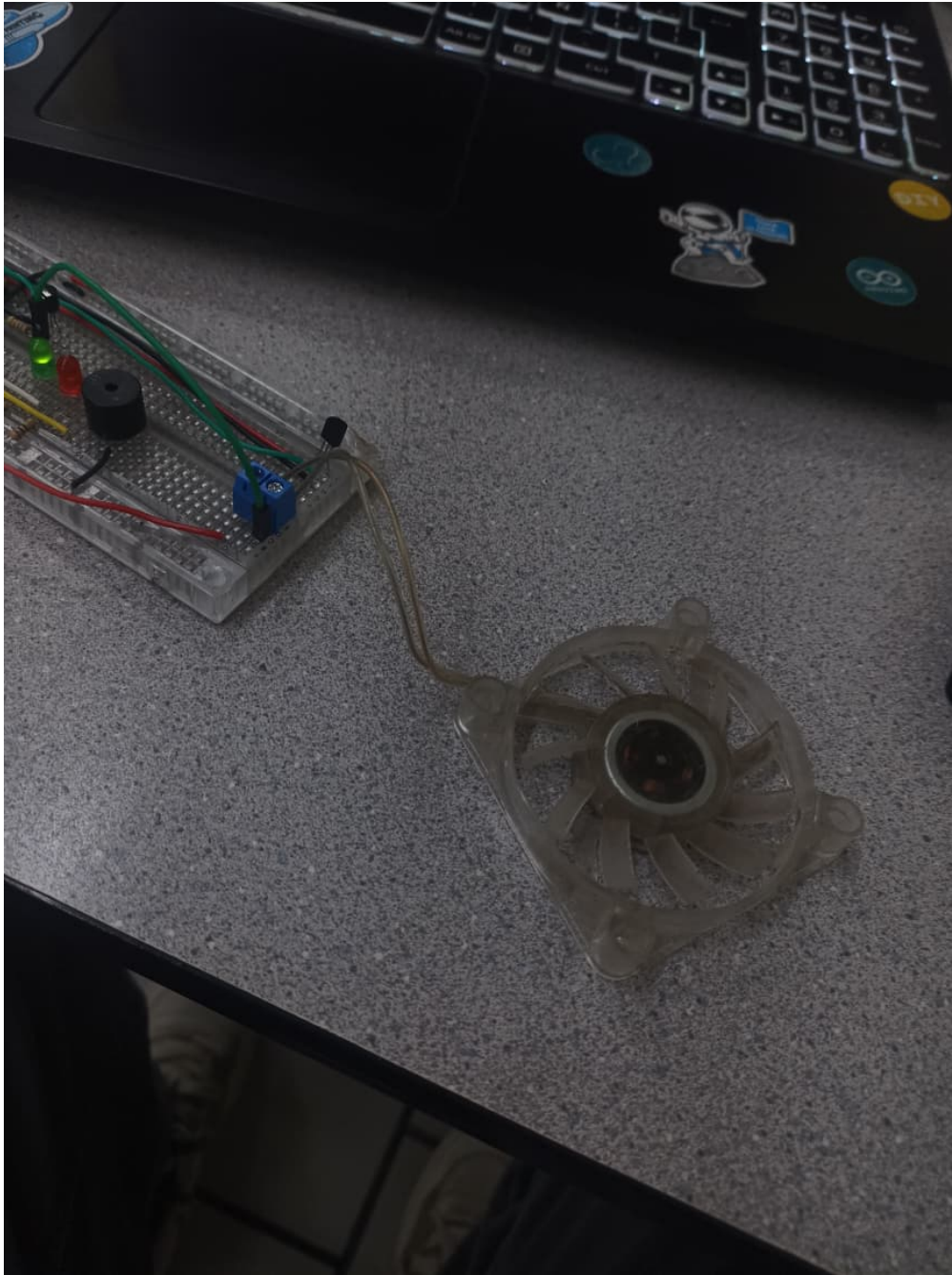


Figura 2: Prototipo físico del circuito de medición de temperatura y control de ventilador armado en protoboard.



Figura 3: Monitor gráfico de temperatura ejecutándose en la computadora, recibiendo datos del Raspberry Pi Pico W vía USB-serial.

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.2). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [4] Texas Instruments, *LM35 Precision Centigrade Temperature Sensors* (Rev. H). <https://www.ti.com/lit/ds/symlink/lm35.pdf>, 2023.
- [5] ON Semiconductor, *2N2222A NPN Silicon Switching Transistor*. <https://www.onsemi.com/pub/Collateral/P2N2222A-D.PDF>, 2023.

- [6] pySerial Contributors, *pySerial Documentation*. <https://pyserial.readthedocs.io>, 2024.
- [7] Matplotlib Development Team, *Matplotlib: Visualization with Python*. <https://matplotlib.org/stable/>, 2024.